SensorMind: An Industrial IoT Sensor Intelligence Platform

Abstract: SensorMind is an Industrial IoT sensor intelligence platform. It shows the equipment of a site in a 3D floor view, with live sensor values and warnings beside each machine. An assistant answers questions by combining the live readings with equipment manuals, and it names the manual page it used. This report explains what the platform does, which technology it uses, and how a customer's sites, floors, machines and sensors are organised. Supermarket fridges serve as the example. All sensor data come from a built-in simulator, and the assistant was tried only against a local stand-in for the AI service. No benefit was measured, and nothing was tested in a real store.

1. What SensorMind Is

SensorMind is an Industrial IoT sensor intelligence platform. IoT means the Internet of Things: machines fitted with sensors that report their state. A user signs in, sees the machines of a site in a 3D floor view and reads the live value of each sensor. Every value is marked normal, warning or alarm, so a problem stands out.

The running example is Walnut, a fictional supermarket chain with a store in Berlin. The store has 20 fridge and freezer cabinets on two floors. Each cabinet has three sensors: cabinet temperature, compressor current and door open time. An assistant answers questions such as "Why is cabinet R-03 getting warmer?" It reads the live values and the equipment manual, and it tells the user which manual page it used. The sensor values are simulated and the manual is a fictional demo document, so no real store stands behind the example.

Why it matters. A fridge that drifts warm is easier to deal with when the warning, the readings and the likely causes are in one place.

2. Technology Used

Table 1 lists each part and what it does; Figure 1 shows how the parts connect. The browser shows the screens. The server applies the rules and talks to three outside services: a database, a search store for manuals and an AI service.

Table 1. Technology used and what each part does.

Layer	Technology	What it does
Screens	React, TypeScript	Build the pages and update them as new values arrive
3D floor view	Three.js	Draws the floor and the machines in 3D
Charts	Recharts	Draws trend lines
Styling	Tailwind CSS	Sets colours, spacing and layout
Server	ASP.NET Core 8, API controllers	Holds the rules, such as when a value is a warning, and answers requests from the screens
Customer data	PostgreSQL, Entity Framework Core	PostgreSQL is a database for customers (tenants), sites, users and a request log. Entity Framework Core is the code that reads and writes it
Manual search	Qdrant	A vector database: a database that finds text by meaning
AI models	OpenRouter	A service that gives access to AI models: one turns text into a searchable form, one writes answers
Live readings	Server-Sent Events	The server keeps pushing new values to the browser, with no page reload
Sensor values	Built-in simulator	Produces fake readings, including drifts and alarms
Local services	Docker Compose	Starts the database and Qdrant on a developer's computer
API documentation	Swagger	Lists the server's endpoints and lets a developer try them

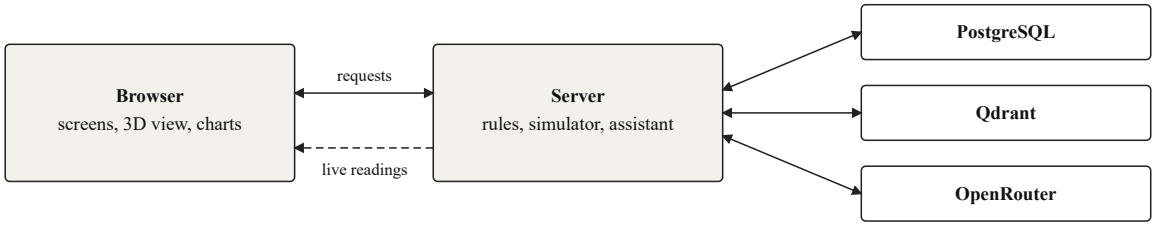


Figure 1. How the parts connect. The dashed arrow is the stream of live readings. PostgreSQL holds customer data, Qdrant holds the manual passages, and OpenRouter provides the AI models.

3. Parts of the Application

After signing in (Figure 3a) the user sees the main view (Figure 2). It has an asset tree on the left, the 3D floor in the middle, and the sensor panel and assistant on the right. To work with it, pick a floor with the tabs above the 3D view, then click a

machine (an asset) in the tree or in the 3D view. The sensor panel lists the machine's sensors (tags) with the current value, the status and a short trend line. The user can then ask the assistant a question (Figure 6).

Warnings pulse slowly in amber and alarms pulse faster in red in the 3D view, the tree, the floor tabs and the notification bell. A still image cannot show the pulse, so the figures show the colours only. The bell (Figure 3b) lists each new warning and alarm with its floor and value. The list exists only in the browser and is empty after a reload.

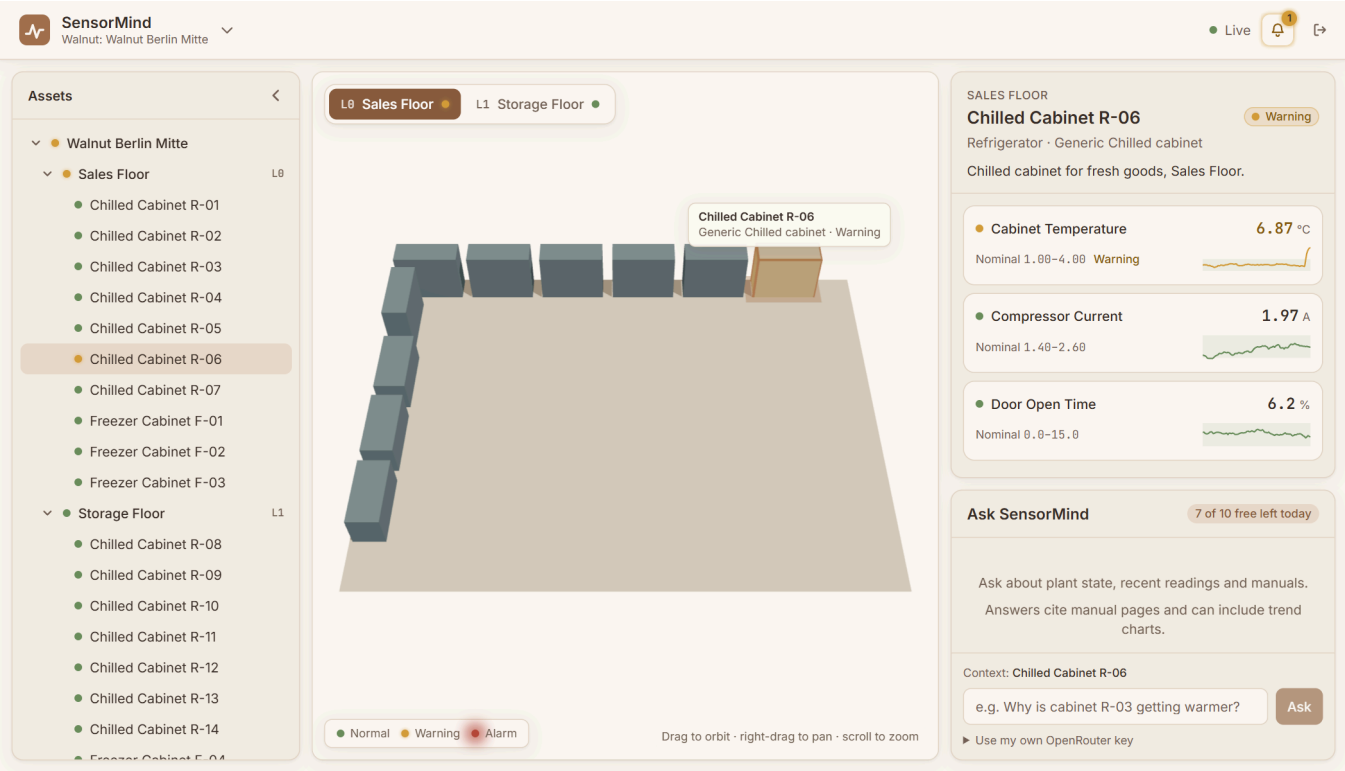
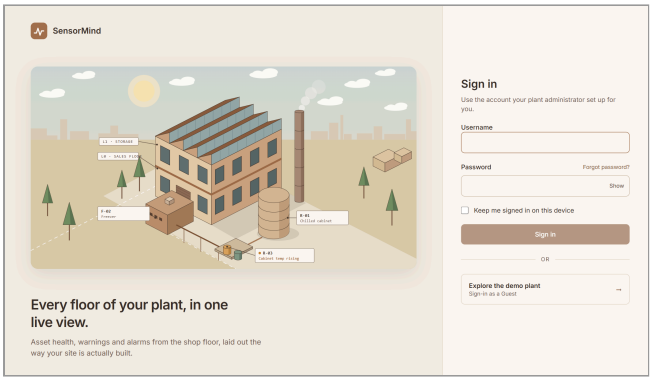
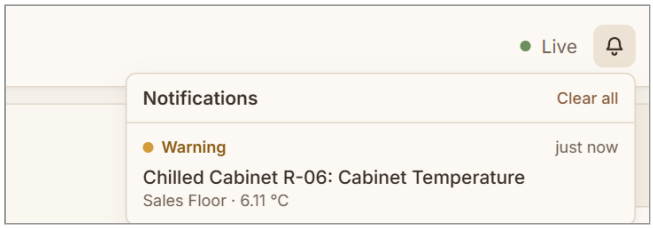


Figure 2. Main view with Chilled Cabinet R-06 selected. It shows a warning (6.87 °C, above the 4 °C limit) that the simulator produced on its own; nothing was forced. Amber dots mark the cabinet, its floor and the site, and the cabinet is tinted amber in the 3D view.



(a)



(b)

Figure 3. (a) Sign-in page, with a button for a guest account. (b) Notification list with a warning that the simulator raised on its own.

4. How a Customer Is Organised

Everything is arranged in five levels (Figure 4). A tenant is a customer company, here Walnut, and it holds one or more sites. A site is a place such as a store: "Walnut Berlin Mitte". A site has floors, a floor holds assets (the machines), and an asset has tags (the sensor signals). Assets sit directly under a floor, with no groups in between. The tenant is stored in the database together with its sites. The floors, assets and tags of a site are written in a text file that is checked when it is loaded, and every mistake in it is reported.

Each tag has a normal range and, usually, alarm limits. A value inside the range is normal. A value outside it is a warning. A value at or beyond an alarm limit is an alarm. For Cabinet Temperature the normal range is 1 to 4 °C and the alarm limits are 7 °C and −1 °C. An asset, a floor and a site show the worst status found below them.

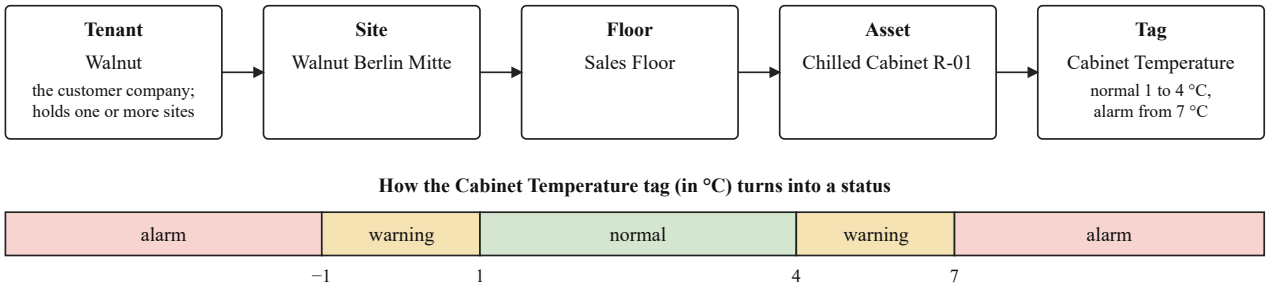


Figure 4. The five levels, with Walnut as the example. Each level holds one or more of the next. The bar below shows how one tag value maps to a status; its scale is schematic.

5. Example: What Happens at Walnut

Table 2 follows one chilled cabinet from a small drift to a likely cause.

Table 2. A cabinet warms up: the steps and what the user sees.

Step	What happens	What the user sees
1	The temperature of cabinet R-03 drifts above 4 °C.	A warning: an amber pulse on the cabinet, its floor tab and the tree, and a new item in the bell.
2	The temperature keeps rising and reaches 7 °C.	An alarm: the pulse turns red and the bell lists the change.
3	The user selects the cabinet.	Its three sensors with values, status and trend lines.
4	The user asks, “Why is cabinet R-03 getting warmer?”	The question appears in the chat panel.
5	The assistant combines the live readings with page 3 of the demo manual. That page lists causes: a door open too long or a worn door seal, blocked airflow, a dirty condenser, a defrost fault, refrigerant loss and a compressor fault. It advises checking the simplest cause first.	An answer that names likely causes and what to check, with a chip citing the manual page.

What this gives. The platform is meant to give earlier notice of a problem, one place to look instead of several tools, and causes and next steps next to the readings. No benefit was measured: all data are simulated and nothing was tested in a real store.

6. How the Assistant Answers

When a user asks a question (Figure 5), the platform does two things. It searches the manuals for passages that match the meaning of the question, and it reads the current values and recent trend of the relevant sensors. An AI model then writes the answer from these two sources and names the manual pages it used. The manuals are prepared once, when the server starts: each page is split into passages, turned into numbers that capture meaning and stored in Qdrant, and a manual already stored is skipped. If the manual search fails, the assistant still answers from sensor values.

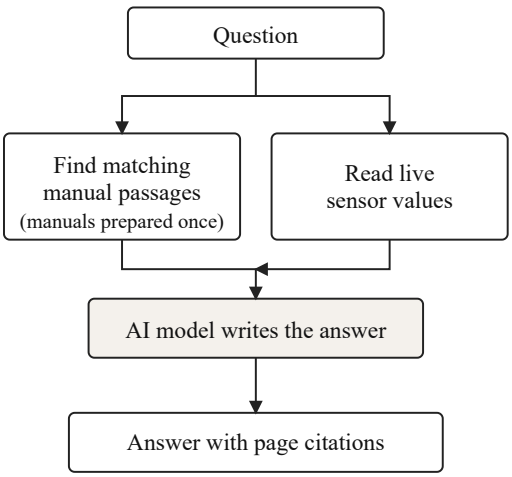


Figure 5. How a question is answered.

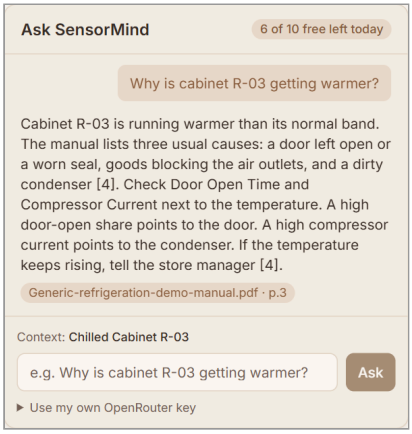


Figure 6. The assistant answering the example question. The text comes from a local stand-in for the AI service and is a fixed reply; the citation chip (page 3 of the demo manual) is the real result of the manual search.

7. Testing and Limits

The platform was run by hand on throwaway copies: signing in, live values, notifications, loading the demo manual and asking a question. There are no automated tests. All data are simulated and no real device is connected. The assistant was tried only against a local stand-in for the AI service, so answer quality, speed and cost with a real model are not known. The

demo manual is fictional and was written for this project. Alerts exist only in the app, not by email or text message, and are lost on reload. Accounts are not tied to a tenant, and no administrator account is created automatically, so one must be added to the database by hand. Future work is to connect real sensors, independently of the equipment controllers, to add email and text alerts, and to evaluate answers with real manuals.

8. Conclusion

SensorMind brings sensor values, alarms, a 3D floor view and a manual-aware assistant into one platform, shown here with a fictional supermarket. The prototype runs end to end on simulated data. Whether it helps in a real store is not yet known, because it has not been tested on real equipment or with a real AI model.

Appendix A. AI-Assisted Development Workflow

I built SensorMind with Claude Code, Anthropic's coding assistant, in my editor. The AI wrote most of the code and the first drafts of the documents. The direction, the decisions and the checking were mine.

What I did. I defined the product and its scope: a 3D floor view, live status and alerts, and an assistant that answers from live readings and manuals. I wrote the kickoff specification, which fixes the technology (for example .NET 8, React, Three.js, Qdrant and OpenRouter) and splits the work into four phases. I made the later design decisions and recorded them in *docs/decisions.md*: Entity Framework Core with the repository pattern for all database access, API controllers instead of minimal APIs, no group level, "asset" instead of "equipment", the tenant as the top level, the columns of the request-log table, the fictional Walnut supermarket in place of the earlier sample plant, and no automated tests. I set the working rules in *CLAUDE.md* (short answers, no changes until I say start, minimal code) and saved a preference that the AI never commits or pushes; I make every commit myself. I kept a task list and gave each task to its own agent, running tasks in parallel only when they did not touch the same files.

How the tool was used. A main session received my instructions, planned, and handed each task to a separate subagent, a custom "task-executor" agent (Figure A1). Each subagent changed the code or documents for one task, tested the result on throwaway databases and ports, never on my own database or running application, and reported what it changed, what it checked and what it could not check. A PostgreSQL MCP server (Model Context Protocol, a standard way to give the AI tool extra abilities) was part of this setup: it let the tool inspect the project's database, its tables and rows, directly while diagnosing problems, instead of through manual Docker commands.

Review and limits. I reviewed every result in the running application and corrected it with screenshots and instructions: the layout that did not fit the screen, the width of the sign-in panel, the illustration (extended, not stretched), the loader, the notification bell, the L-shaped layout, the blinking of warnings and alarms, and the wording of the report and sign-in page. I directed the report itself: its structure, plain wording, what to remove, and the "simulated and fictional" framing. To limit risk, I fixed the product name, took a backup before a database reset, and approved that reset explicitly after the tool's permission check held it back. I held back features that were not worth the effort, such as an icon bank. AI output can be wrong, so I relied on running the application and on my own review. There are no automated tests, and I did not measure effort, time or defect rates, so I make no claim about them.

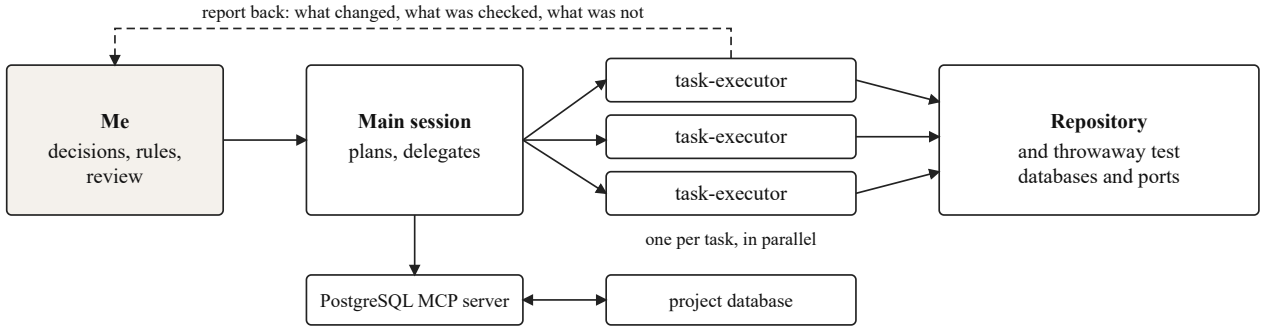


Figure A1. The workflow. I give instructions and decide; each task goes to its own subagent, which works on the repository and on throwaway test instances and reports back to me. The PostgreSQL MCP server lets the main session inspect the project database.